



SESSION 3

MODULES AND PACKAGES IN NODE.JS

Learning Objectives

In this session, students will learn to:

- Explain what modules are and its three types
- Explain the significance of packages
- Describe the purpose of Web framework and utility functions

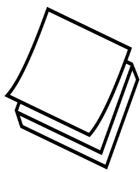
Modules are a ready-to-use, time-tested, reusable collection of functions or methods that can be used by other applications to make the process of development easier.

Packages are a collection of one or more modules grouped together as a single unit that can be published and installed in Node.js applications. Web frameworks are a collection of libraries and tools that provide specific features. Using Web frameworks in Node.js applications simplifies the development process and reduces the necessity to repeat the same code in multiple places.

This session will explain different types of modules and Web frameworks that can be used to build Node.js applications. It will explain how to install and use packages. Finally, it will discuss the importance of utility functions.

3.1 Modules in Node.js

Modules can be thought of as an application where functions or methods are wrapped up together. They are available as a ready-to-use instant application. Modules must be imported into the code to use the functions defined in them. The advantage of modules is that once defined, they can be reused inside any number of Node.js program files.



A method is also a function that is associated with an object. However, a function is not associated with any object. For example, a GET method can be defined for a HTTP Request or Response object. Require is a Node.js function that is used to import modules in a Node.js program.

Different types of modules in Node.js are follows:



3.1.1 Core Modules

Built-in modules are referred to as core modules, which are part of the Node.js platform. The built-in modules get installed along with the Node.js installation.

Table 3.1 lists some of the core modules.

Name of the Module	Description
http	Provides methods to create an HTTP server in Node.js.
assert	Provides methods with a set of assertion functions for testing.
fs	Provides methods to handle file system.
path	Provides methods to deal with file paths.
process	Provides methods to get information and control about the current Node.js process.
os	Provides methods to get information about the operating system.
querystring	Provides methods for parsing and formatting URL query strings.
url	Provides methods for URL resolution and parsing.

Table 3.1: Built-in Modules

3.1.2 Local Modules

A local module is a .js file that includes user-defined functions within it. It is created locally within a Node.js application and can be distributed across projects using the Node Package Manager (NPM). For example, a local module can be created to contain a function that returns the current date and time and can be distributed across projects.

3.1.3 Third-Party Modules

Third-party modules are modules that are not native to Node.js. They are created by some third party for use by other Web applications and are available online. They can be installed using the NPM.

Table 3.2 lists some of the modules of third-party modules.

Name of the Module	Description
Mongoose	It is a MongoDB Object data modeling library. It converts code from MongoDB to a Node.js server.
Express	It is a library for building Web and mobile applications. It manages the routing of a server.
Angular	It is a library to build robust Web applications. It is based on the MVC model.
React	It is a library for building the front-end interface of applications. It aids in creating creative and complex user interfaces.

Table 3.2: Third-Party Modules

An advantage of using a third-party module is that there is no overhead for writing the code and testing it. It is tested and available for instant use.

3.2 Working with Modules

Node.js provides the `require` function for importing modules. The syntax for using the `require` function is as follows:

```
Var_module_name = require('module_name')
```

Here, `Var_module_name` is the name of the variable to which the instance of the imported module is assigned. `module_name` is the name of the module that is being imported such as `os`, `http`, and `fs`.

Code Snippet 1 shows an example code that imports the built-in module `fs` using the `require` function and assigns it to the variable named `Var_fs`. The code then prints a message to the console.

Code Snippet 1:

```
var fs=require('fs');  
console.log('An instance of File system object created');
```

To execute the code, type the code in VS Code and save it with the file name `code-snippet1.js`. Then, execute the `.js` file using the command as in:

```
node code-snippet1.js
```

Figure 3.1 shows the output of the execution of Code Snippet 1.

```
C:\Users\write\Sample code>node code-snippet1.js  
An instance of File system object created
```

Figure 3.1: Execution of `code-snippet1.js`

After importing the module into the Node.js code, start using the methods in the module by accessing them with the instance of the module. For example, the `fs` module provides a set of methods to perform the given actions on a file:

- Opening a file
- Writing to a file
- Appending a file
- Reading a file
- Deleting a file

Table 3.3 describes these methods.

Method	Description
<code>fs.open</code>	Performs the specified operation on the specified file such as read, write, and append. If the file does not exist, creates a new file. Opens the file in the operation mentioned such as read or write or read and write. Creates the file if it does not exist.
<code>fs.writeFile</code>	Performs the write operation on a file. Creates a new file if the file does not exist. Overwrites the file if it already exists.
<code>fs.appendFile</code>	Appends the given contents to a file if it already exists. Else, creates a new file and appends the contents to it.
<code>fs.readFile</code>	Reads the contents of a file.
<code>fs.unlink</code>	Removes the file from the directory.

Table 3.3: File Handling Methods

3.2.1 Opening a File

The syntax for opening or creating a new file using the `open` method is as follows:

```
fs.open(path, flags[,mode], callback)
```

Here:

- `path` denotes the path of the file to open.
- `flags` denotes the operation for which the file is to be opened. The flags can be used to open the file for the read or write operation.
- `mode` specifies the mode of the file in which it has to be opened.
- `callback` is a callback function that is called after the file opening operation is complete.

Code Snippet 2 shows the code to create a file using the `fs.open` method. The code creates an empty text file named `filecontent.txt` in write mode.

Code Snippet 2:

```
var fs = require('fs');  
fs.open('filecontent.txt', 'w', function (err) {  
    if (err) throw err;  
    console.log("File created successfully.")  
});
```

In Code Snippet 2, `function(err)` is a parameter of the `fs.open` method. This is the callback function which is called if an error occurs as the `err` parameter is specified. This callback function throws the error specified by the `err` parameter. If no error occurs when opening the file, the File created successfully message is written to the console. Type the code in VS Code and save the file as `openfile.js`.

Figure 3.2 shows the output of the execution of the file.

```
C:\Users\write\Sample code>node openfile.js  
File created successfully.
```

Figure 3.2: Execution of `openfile.js`

Figure 3.3 shows the contents of `filecontent.txt`, opened using Notepad.

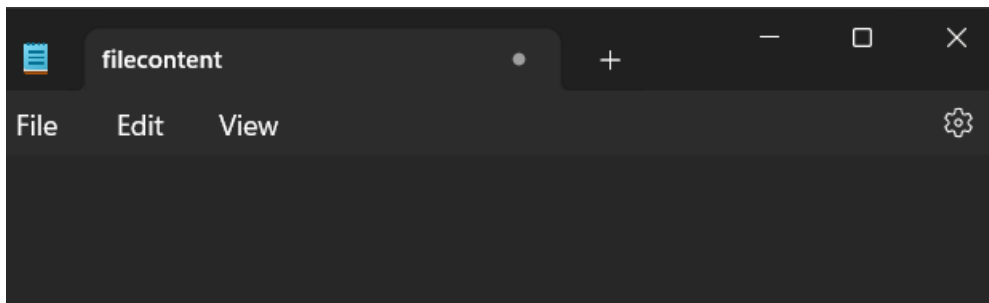


Figure 3.3: Contents of the filecontent.txt File

3.2.2 Writing to a File

The syntax for creating a file using the `writeFile` method is:

```
fs.writeFile(filename, data, [encoding],  
             [callback_function])
```

Here:

- `filename` denotes the name of the file.
- `data` is the content to be written to the file.
- `encoding` is an optional parameter that represents the way the data is represented inside the file system. ASCII, Base64, UTF64 are the common encoding formats. UTF64 is a commonly used encoding format for representing text files internally.
- `callback_function` is an optional parameter that specifies a callback function, which is called after the contents are written to the file. This function takes two parameters:
 - `err`: Is an error object that contains the error message if any error occurs.
 - `data`: Is a buffer or string that contains the file content.

Code Snippet 3 shows the code to create a file named `nodefile.txt` and write a custom message into it.

Code Snippet 3:

```
var fs = require('fs');  
fs.writeFile('nodefile.txt', 'Welcome to Node.js  
Programming. In this session, we learn about File  
System module.', function (err) {  
  if (err) throw err;  
  console.log('File is written successfully.');
```

```
});
```

In Code Snippet 3, the filename, data to be written, and the callback function are passed as parameters to the `writeFile` method. If an error occurs, the callback function is called by passing the error code, `err`. If the method executes successfully, then a message is displayed on the console.

Type the code in VS Code and save it with the file name `createfile.js`. Execute the code. Figure 3.4 shows the output of the execution of code snippet 3.

```
C:\Users\write\Sample code>node createfile.js
File is written successfully.
```

Figure 3.4: Execution of `createfile.js`

To verify that the text file is created, type the `dir *.txt` command at the command prompt. Find the file listed in the current directory.

Figure 3.5 shows the `nodefile.txt` file listed in the current directory.

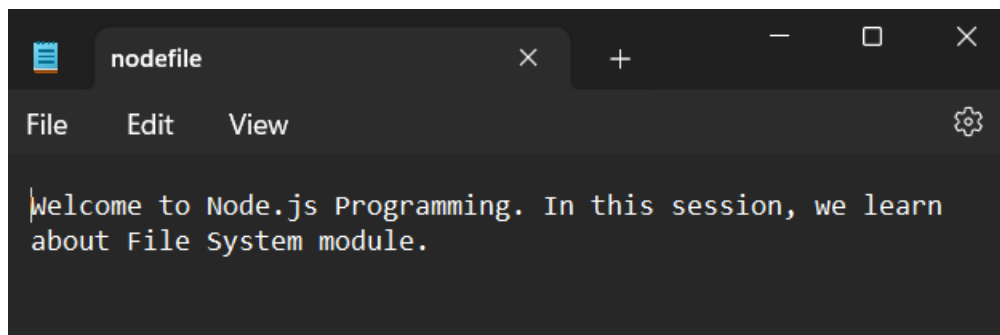
```
C:\Users\write\Sample code>dir *.txt
Volume in drive C is Windows-SSD
Volume Serial Number is 26B1-8262

Directory of C:\Users\write\Sample code

09-11-2023  15:16                0 filecontent.txt
03-11-2023  07:33            136 in the post..txt
09-11-2023  16:22            83 nodefile.txt
13-10-2023  05:19                0 openfile.txt
13-10-2023  11:34           160 readfile.js.txt
13-10-2023  15:09            42 sampleopen.txt
13-10-2023  11:44            93 sampleread.txt
13-10-2023  04:25            51 samplewrite.txt
               8 File(s)              565 bytes
               0 Dir(s)  74,353,127,424 bytes free
```

Figure 3.5: File Listed in the Directory

Figure 3.6 shows the contents of the `nodefile.txt` file.



The screenshot shows a text editor window with the title bar 'nodefile'. The menu bar includes 'File', 'Edit', and 'View'. The text content of the file is: 'Welcome to Node.js Programming. In this session, we learn about File System module.'

Figure 3.6: Contents of the File `nodefile.txt`

3.2.3 Appending to a File

Appending is adding contents to the end of the file. The `appendFile` method adds the given data to the end of a file. If the file does not exist, it creates a new file.

The syntax for appending a file using the `appendFile` method is:

```
fs.appendFile(filename, data, [options], callback)
```

Here:

- `filename` denotes the name of the file.
- `data` is the content to be written to the file.
- `options` is an optional parameter that represents encoding, mode, or flag.
- `callback_function` is an optional parameter that specifies a callback function, which is called after the contents are written to the file. This function takes two parameters:
 - `err`: Is an error object that contains the error message if any error occurs.
 - `data`: Is a buffer or string that contains the file content.

Code Snippet 4 shows the code to append a file.

Code Snippet 4:

```
var fs = require('fs');
var data="Here we create,read and write contents to the
file."
fs.appendFile('nodefile.txt', data, function (err)
{
    if (err) throw err;
    console.log("File Appended successfully.")
});
fs.readFile('nodefile.txt', function(err, data)
{
    if (err) throw err;
    console.log(data.toString('utf8'))
});
```

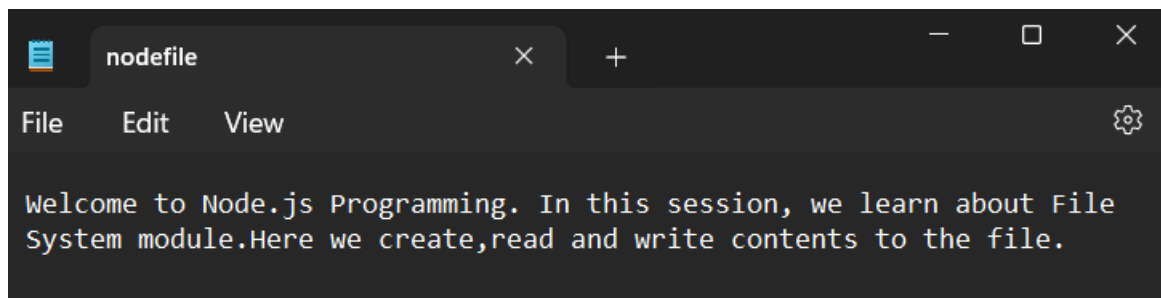
In Code Snippet 4, the `appendFile` method appends the string content stored in the variable named `data` to the `nodefile.txt` file. The `readFile` method then, reads and displays the contents of the `nodefile.txt` file to the console.

Save the code in a file named `appendfile.js` and execute the file. Figure 3.7 shows the output of the execution. The output displays contents of the `nodefile.txt` file with the appended contents.

```
C:\Users\write\Sample code>node appendfile.js
Welcome to Node.js Programming. In this session, we learn about File System module.Here we
create,read and write contents to the file.
File Appended successfully.
```

Figure 3.7: Execution of `appendfile.js`

Figure 3.8 shows the content of the text file `nodefile.txt` where the text data is appended.



```
nodefile
Welcome to Node.js Programming. In this session, we learn about File
System module.Here we create,read and write contents to the file.
```

Figure 3.8: Contents of the File `nodefile.txt`

3.2.4 Reading a File

To read the contents of the text files, use the `readFile` method of the `fs` module.

The syntax of the `readFile` method is:

```
fs.readFile(filename, callback)
```

Here, `filename` specifies the path of the file to read. `callback` is the callback function that is called after the reading operation is complete.

Code Snippet 5 shows the code to read a file.

Code Snippet 5:

```
var fs = require('fs');
fs.readFile('nodefile.txt', function (err) {
    if (err) throw err;
    console.log("File read successfully.")
});
```

The code reads a text file called `nodefile.txt`.

To execute the code, save the code with the file name `readfile.js` and execute the file. Figure 3.9 shows the output of the execution. Contents of the `nodefile.txt` file are displayed in the console.

```
C:\Users\write\Sample code>node readfile.js
Welcome to Node.js Programming. In this session, we learn about File System module. Here we
create, read and write contents to the file.
```

Figure 3.9: Execution of `readfile.js`

3.2.5 Deleting a File

The `fs.unlink` and `fs.rmdir` methods help in deleting a file.

The syntax for deleting a file using the `unlink` method is:

```
fs.unlink(filename, callback)
```

Here, `filename` denotes the path of the file to delete. `callback` is an optional parameter that specifies a function to be called after the delete operation is complete.

Code Snippet 6 shows the code to delete a file.

Code Snippet 6:

```
var fs = require("fs");
fs.unlink('filecontent.txt', function(err) {
  if (err) throw err;
  console.log("File deleted successfully!");
});
```

The code calls the `unlink` method to delink the specified file from the current directory thereby deleting it from the directory.

Save the code with the filename `deletedefile.js`. Execute the file. The file `filecontent.txt` is used as a sample file for the deletion process. Once the code executes, the file gets deleted. Figure 3.10 shows the execution of the Code Snippet 6.

```
C:\Users\write\Sample code>node deletedefile.js
File deleted successfully!
```

Figure 3.10: Execution of `deletedefile.js`

Figure 3.11 shows the execution of the `dir *.txt` command to show the listing of the `filecontent.txt` file in the current directory before and after the execution of the code in `deletefile.js`.

```
C:\Users\write\Sample code>dir *.txt
Volume in drive C is Windows-SSD
Volume Serial Number is 26B1-8262

Directory of C:\Users\write\Sample code

09-11-2023  16:41                0 filecontent.txt
03-11-2023  07:33            136 in the post..txt
09-11-2023  18:21            134 nodefile.txt
13-10-2023  05:19                0 openfile.txt
13-10-2023  11:34            160 readfile,js.txt
09-11-2023  17:42            195 sampleopen.txt
13-10-2023  11:44             93 sampleread.txt
13-10-2023  04:25             51 samplewrite.txt
               8 File(s)              769 bytes
               0 Dir(s)  74,326,167,552 bytes free

C:\Users\write\Sample code>node deletefile.js
File deleted successfully!

C:\Users\write\Sample code>dir *.txt
Volume in drive C is Windows-SSD
Volume Serial Number is 26B1-8262

Directory of C:\Users\write\Sample code

03-11-2023  07:33            136 in the post..txt
09-11-2023  18:21            134 nodefile.txt
13-10-2023  05:19                0 openfile.txt
13-10-2023  11:34            160 readfile,js.txt
09-11-2023  17:42            195 sampleopen.txt
13-10-2023  11:44             93 sampleread.txt
13-10-2023  04:25             51 samplewrite.txt
               7 File(s)              769 bytes
               0 Dir(s)  74,327,367,680 bytes free
```

Figure 3.11: Directory Listing

3.3 Packages

A package is a container of modules. A package groups all related modules together as a single unit so that they can be imported into applications.

To import a package into your application, use the `require` function. The functions within the modules of a package can be accessed using the instance that is returned by the `require` function.

3.3.1 Installation of Packages

To install packages, use the `npm` command-line tool. This command-line tool is installed with the Node.js installation. To verify the installation of `npm`, type the command `npm -v` at the command prompt.

Figure 3.12 shows the output of verifying the version of the `npm` tool.

```
C:\Users\write>npm -v
10.1.0
```

Figure 3.12: Verifying the Version of `npm`

To update an older version of `npm` to the latest version, use the command as in:

```
npm install npm -g
```

Here,

- `npm` is the name of the command-line tool that runs the command.
- `install` is the command that installs the specified package.
- `g` is the flag to denote that `npm` should be installed globally on the system, rather than locally in the current project directory.

Install Packages Locally

To install packages locally in the current project directory, use the command as in:

```
npm install <package_name>
```

Here, `package_name` is the name of the package to be installed. For example, the command to install Angular CLI locally in your current project directory is:

```
npm install @angular/cli
```

Figure 3.13 shows the execution of this command at the command prompt and the installation of Angular packages.

```
C:\Users\write>npm install @angular/cli  
  
added 244 packages in 1m
```

Figure 3.13: Installation of the Package 'angular'

Locally installed packages are available only to the Node.js applications present within the respective local directory. By default, NPM places the modules of a package into the local `node_modules` directory. To verify, type `dir` at the command prompt. Find the `node_modules` directory listed as shown in Figure 3.14.

```
C:\Users\write>dir  
Volume in drive C is Windows-SSD  
Volume Serial Number is 26B1-8262  
  
Directory of C:\Users\write  
  
09-11-2023  18:39    <DIR>          .  
06-03-2023  20:28    <DIR>          ..  
09-05-2021  13:16    <DIR>          .android  
14-09-2021  11:25    <DIR>          .fcc  
19-10-2023  10:55    <DIR>          .ipynb_checkpoints  
28-06-2023  19:28    <DIR>          .ipython  
07-08-2023  09:40    <DIR>          .jupyter  
03-09-2023  10:26    <DIR>          .matplotlib  
21-09-2023  12:49          13 .node_repl_history  
27-09-2023  11:25    <DIR>          .vscode  
01-12-2020  18:39    <DIR>          3D Objects  
02-12-2020  13:18    <DIR>          ansel  
09-11-2023  17:25          381 appendfile.js  
19-10-2023  10:53          3,698 code.txt  
06-03-2023  20:39    <DIR>          Contacts  
07-08-2023  11:52          366 create_table.py  
09-11-2023  18:38          151 deeltefile.js  
09-11-2023  18:40          151 deletefile.js  
03-04-2023  18:09    <DIR>          Documents  
09-11-2023  12:02    <DIR>          Downloads  
06-03-2023  20:39    <DIR>          Favorites  
28-08-2023  19:18          417 filextest.pkl  
28-08-2023  19:18          945 filextrain.pkl  
28-08-2023  19:18          241 fileytest.pkl  
28-08-2023  19:18          285 fileytrain.pkl  
19-07-2023  15:46    <DIR>          flask_app  
17-06-2023  17:09          2,578,580 get-pip.py  
06-03-2023  20:39    <DIR>          Links  
06-03-2023  20:39    <DIR>          Music  
09-11-2023  20:08    <DIR>          node_modules  
09-11-2023  05:56    <DIR>          OneDrive  
09-11-2023  20:08          128,278 package-lock.json  
09-11-2023  20:08          187 package.json
```

Figure 3.14: node_modules Directory

Install Packages Globally

To enable Node.js applications present across the system to access a package, install the package globally by turning on the global mode. Append `-g` to the `npm install` command. For example, to install Angular CLI globally on the system, use the command as in:

```
npm install -g @angular/cli
```

3.3.2 Update Packages

To update a package, use the `npm update` command as in:

3.3.3 Uninstall Package

```
npm update <packagename>
```

To uninstall a package, use the `npm uninstall` command as in:

```
npm uninstall <packagename>
```

Figure 3.15 shows the execution of this command at the command prompt and the uninstallation of Angular packages.

```
C:\Users\write>npm uninstall @angular/cli  
  
removed 244 packages, and audited 1 package in 1s  
  
found 0 vulnerabilities
```

Figure 3.15: Uninstalling a Package

3.4 Web Framework

A Web framework is a set of libraries or tools used for the speedy development of Web applications. They eliminate the requirement to write repetitive code. Frameworks act as a template to design the User Interface (UI) of Web applications. They also provide libraries for database access, session management, and other common tasks. Figure 3.16 shows some of the frameworks used with Node.js.

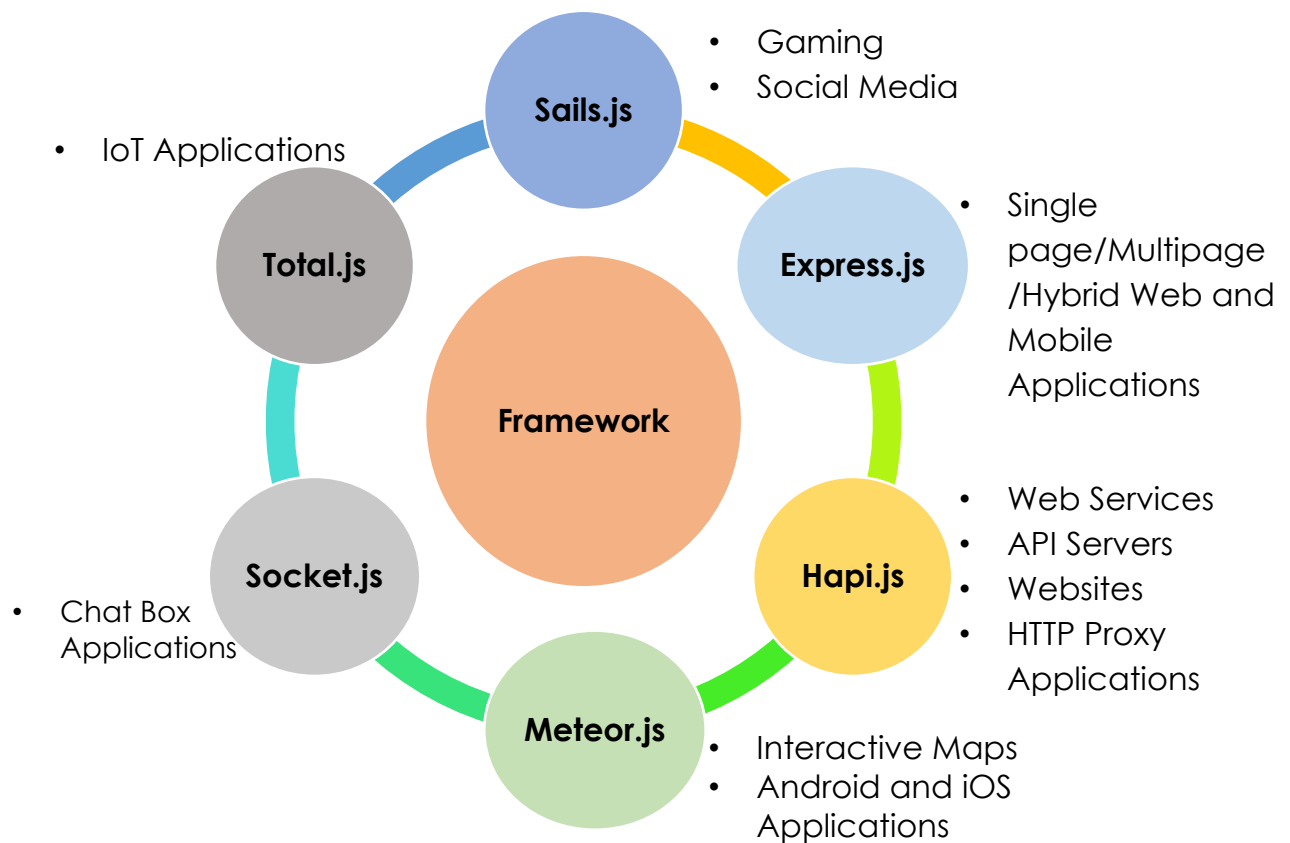


Figure 3.16: Frameworks of Node.js

A node framework is a Web framework that supports the development of applications using Node.js. Node frameworks allows developers to use JavaScript to develop both front-end and back-end of a Web application. There are three types of framework models in Node.js. They are as depicted in Figure 3.17.

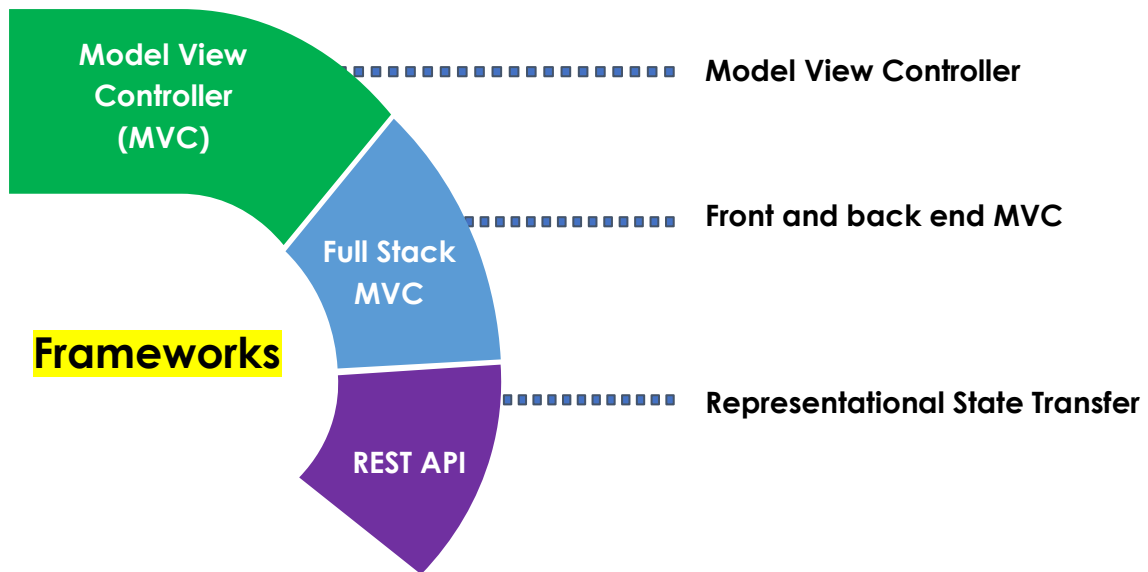


Figure 3.17: Types of Framework Models

3.4.1 MVC Framework

MVC stands as an abbreviation for the term, Model-View-Controller. This model splits the code logic into three parts, namely Model, View, and Controller. The Model component contains the data and business logic. The View component contains the logic to handle the user interface and the output. The Controller component contains the logic to handle the user inputs and coordinate with the Model and the View.

Express.js is modelled on the MVC Framework is Express.js. Figure 3.18 illustrates the working of the MVC model.

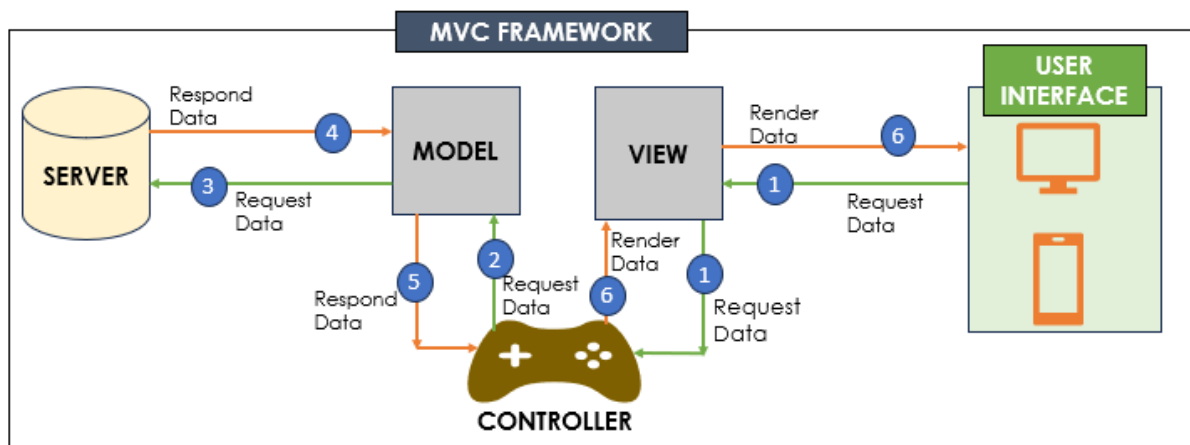


Figure 3.18: MVC Framework

In the MVC Framework model:

1. Users send requests to the Controller through the View.
2. The Controller in turn sends the requests to the Model.
3. The Model requests the database server for the required data.
4. The database server sends the required data as response to the Model.
5. The Model sends the response to the Controller.
6. The Controller in turn uses the View for rendering the response.

Thus, the Controller acts as a bridge between Model and View, controlling the flow of information.

3.4.2 Full Stack MVC Framework

A full stack MVC Framework model supports both front-end and back-end application development. It offers special libraries and tools for the development and management of applications.

3.4.3 REST API Frameworks

REST stands for Representational State Transfer. REST API defines a set of rules for communication between a server and a client using the HTTP methods. By using REST APIs, developers can create and use REST Web services in Node.js applications.

Simplicity and ease of use makes REST accessible to a wide range of developers. Moreover, developers are already familiar with the HTTP methods such as `get`, `put`, `post`, and `delete` that are part of the REST API.

3.5 Utility Functions

Utility functions are general-purpose functions that perform common or basic tasks required for the application development. Node.js provides several utility functions that are wrapped into different utility module as shown in Figure 3.19.

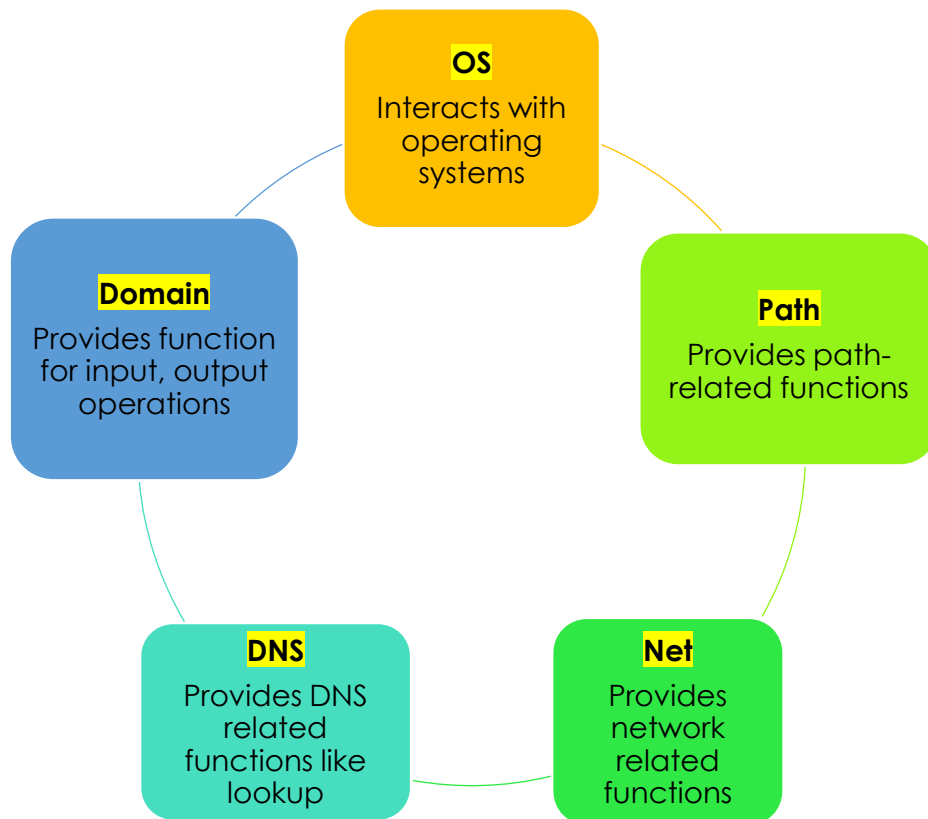


Figure 3.19: Utility Modules

3.5.1 Working with OS Module

Table 3.4 lists the methods of the `os` module.

Name of the Method	Features
<code>os.type</code>	Returns the name of the operating system
<code>os.platform</code>	Returns the platform of the operating system
<code>os.totalmem</code>	Returns the size of total system memory in bytes

Table 3.4: Methods of OS Module

Code Snippet 7 shows the code to print the operating system name, the platform, and the total amount of system memory in bytes using the methods of the `os` module.

Code Snippet 7:

```
const os = require('os');
console.log('Operating System type : ' + os.type());
console.log('Platform : ' + os.platform());
console.log('Total memory : ' + os.totalmem() + '
bytes.');
```

Save the code in a file name `osmethod.js` and execute the file.

Figure 3.20 shows the output of this code.

```
C:\Users\write\Sample code>node osmethod.js
Operating System type : Windows_NT
Platform : win32
Total memory : 8343445504 bytes.
```

Figure 3.20: Execution of `osmethod.js`

3.5.2 Working with Path Module

Table 3.5 lists the methods of the `Path` module.

Name of the Method	Features
<code>path.join(path1,path2,...)</code>	Joins the given strings and creates a new string that can be used as a directory or path
<code>path.parse(path)</code>	Returns an object that provides the path-related details such as root, directory, filename, and file extension
<code>path.dirname(path)</code>	Returns the directories under the given path
<code>path.extname(filename or path)</code>	Returns the extension of a file or file in the path

Table 3.5: Methods of Path Module

Code Snippet 8 shows the code that uses the `Path` module. The code uses the `path.parse` method to display information about the root, base, directory, filename, and extension of file paths `/nodejs/sample code/sampleread.txt` and `/nodejs/sampleread.js`.

Code Snippet 8:

```
const path = require('path');
path1 = path.parse('/nodejs/sample
code/sampleread.txt');
console.log(path1);
path2 = path.parse('/nodejs/sampleread.js');
console.log(path2);
```

Figure 3.21 shows the output of execution of Code Snippet 8.

```
C:\Users\write\Sample code>node pathmethod.js
{
  root: '/',
  dir: '/nodejs/sample code',
  base: 'sampleread.txt',
  ext: '.txt',
  name: 'sampleread'
}
{
  root: '/',
  dir: '/nodejs',
  base: 'sampleread.js',
  ext: '.js',
  name: 'sampleread'
}
```

Figure 3.21: Information About Files

3.6 Summary

- Modules are ready-to-use collections of functions.
- Three types of modules are core modules, local modules, and third-party modules.
- Core modules are built-in modules that come with the installation of Node.js.
- Packages are collections of one or more modules, which are installed using the NPM tool.
- Frameworks are a collection of libraries and tools that speed up the application development.
- Utility functions perform general-purpose tasks that can be useful for communicating with operating systems and working with files and directories.

Test Your Knowledge

1. You are a Web developer in a pharmaceutical company. Which module will you use to create a Web server for handling HTTP requests?

- a) fs
- b) http
- c) os
- d) None of these

2. Which of the following are built-in modules in Node.js?

- a) assert
- b) querystring
- c) express
- d) mongoose

3. What will be output of the given code?

```
var fs=require('fs');  
console.log('Packages are a collection of modules');
```

- a) Displays Nothing
- b) Displays fs module is not recognized
- c) Shows error in the code
- d) Displays Packages are a collection of modules

4. You are working on a file management project as a lead developer. You want to check if a file already exists and overwrite the data in it. If the file does not exist then, you want to create a file and write the data to it. Which method will you use to perform the given scenario?

- a) fs.writeFile
- b) fs.appendFile
- c) fs.open
- d) None of these

5. You are required to communicate with a remote server to fetch and send data. Which framework will you choose to code without worrying about the network issues?

- a) REST API Frameworks
- b) MVC Framework
- c) Full Stack MVC Framework
- d) Other Framework

Answers to Test Your Knowledge

1	b
2	a, b
3	d
4	a
5	a

Try It Yourself

1. You are working on an admin file management application. Your application must create admin profile, update admin information, read admin data, append new information, and delete admin profile. Write a program using the file system module.
 - a) Create a file named `Admin.txt` and write the text `Admin profile created successfully` into it.
 - b) Create a file named `adminOpenFile.txt` with the text `Admin file open successfully`. Using a Node.js script, open this text file, read, and display its contents.
 - c) Further, in the script, append the text `Admin data appended successfully` to `admin open file` to the `adminOpenFile.txt` file. Read and display its contents.
 - d) Add code in the script to delete the `adminOpenFile.txt` and display a message `Admin File deleted successfully`.
2. Write a program to display the operating system, platform of the operating system, and size of total system memory in bytes.
3. Write a program to display all the directories in a specified path.
4. Write a program to display the version of `npm` installed in your system.